Lab 4 – Auto TileMap Navigation using Graph [100 Marks]

A Partially Autograded Evaluation with Project Integration

IMPORTANT: **Your submission must compile without syntactic error to receive grades. Non-compilable solutions will not be graded.**

IMPORTANT: **You must upload your TileGraph.txt to the *Avenue Dropbox* before the deadline to receive grades.**

This lab was designed mostly by Prof. Scott Chen.

Lab Deadline

The deadline for Lab 4 is Saturday, March 14, 2026, at 11:59 pm.

Deliverables

You must upload on or before the posted deadline the following item to the Avenue dropbox:

- TileGraph.txt (i.e. change .java extension to .txt)
- You further will demonstrate your Model Code and the correct automatic navigation behaviour for the tracer object.

Premise

Labs 4 and 5 of the Winter 2026 COMPENG 3SM4 are two branches of an automatic pathfinder AI to be used in the vintage classic top-down 2D sandbox game like Grand Theft Auto 1 / 2 from the 90s. Your software team is responsible for designing simple AI logics of an automated vehicle that will always try to navigate in a city map and find the best route from the starting intersection to the "goal" intersection to complete the mission.

Lab 4 – Automated Graph Generation from BitMap, and Basic Shortest Path Finding Algorithm

Part of your software team has laid the groundwork of creating the basic tile-map generation logic for the game, so that the game sees a different map generated randomly every time the game is launched. Your job is to design a class (TileGraph), where each object represents a **directed, unweighted graph** constructed based on a tile map. The vertices of the graph are the intersections in the tile map and the edges are roads connecting neighbouring intersections from West to East or from North to South. The automated vehicle class, Tracer, will use the class TileGraph to plan the route from the starting intersection to the goal intersection that passes through the least number of intersections along the way.

Lab 5 – Automated Path Optimization using Different Algorithms with Limited Scope of View

In this lab, the software team further optimizes the game by making the map significantly larger, and the view window limited to 50x25 size. This imitates the visual limitation of the AI driver in the game world, hence changing the way the driver chooses the "optimal path" to reach the goal intersection. Each road now has an associated **speed-cap** parameter, which awards or penalizes the driver with increased or reduced vehicle speed,

and ultimately affects how fast the driver completes the mission. To accommodate these changes, you need to update your Graph class to directed and weighted, so that you can create a few different AI models for the driver to optimize its chosen path continuously to maximize its vehicle speed within the 50x25 scope of vision.

The end result of these two labs will be a collection of simple AI driver logic modules that can be interchangeably deployed in the automatic driver AI for future game development.

Objective Overview

Lab 4 focuses on the implementation of an ***Unweighted Directed Graph*** of the most basic building block of the in-game tilemap, namely a Tile. With the automatic tile-map random generator provided, your tasks are to complete the full Graph implementation and the additional interfacing methods to integrate your designed features into the game to drive the auto-driving tracer to find the unweighted shortest path to reach the destination.

The scope of the lab is the following:

- Implementation of the Graph of Tiles using Adjacency List architecture
- Implementation of TileMap to Graph auto-generation using the Breadth-First Search approach
- Implementation of Depth-First Traversal
- Implementation of *Path Finding with Lowest Number of Vertices* using Breadth-First Search approach
- Complete the lab demo of your integrated model code.

Coding Objectives

Analyze and understand the existing objects in the model code – Tile class, TileMap class, Tracer class, and the main model code.

Use the provided test bench **in the same class**, complete all the missing implementations of TileGraph.java.

The TileGraph class uses the existing Tile and TileMap classes. The intersection Tiles (I) on the map are used as vertices to construct the TileGraph.

Model Code Integration Objectives

Once you have fully implemented TileGraph.java (i.e. passing all the test cases), you need to complete the model code integration in the TileGraph class:

- Note that the TileGraph has the second constructor that accepts the TileMap as the initialization input reference. This constructor is invoked by the Tracer class to instantiate its own TileGraph member with the map it resides in. The key method to connect the dot is the buildGraph() method in TileGraph class.
- To complete the buildGraph() method, you need to scan the TileMap and identify all the intersections to be inserted into the TileMap. This can be done using a similar approach as the breadth-first search algorithm used in findShortestPath() method in TileGraph().
- To make your task easier, you should start the scanning process from the Start intersection and radiate the search outward in breadth-first manner towards the Goal intersection.
- Think about how you can replace getAdjacentVertices() method in TileGraph with your own neighbour-finding algorithm in buildGraph().

After completing buildGraph(), you should be able to see the path automatically generated and drawn on the TileMap in the game, and the tracer (P) will automatically follow the generated path to travel to the Goal intersection from the Start intersection.

Marking Scheme

- **[Total 70 Marks] Autograding Section – TileGraph Code Implementation and Behaviour**
  - **[8 Marks]** Implement AddEdge() method – 4 marks
    Pass both testAddEdge test cases – 2 marks
    Create and pass your own testAddEdgeCustom() test case – 2 marks
  - **[6 Marks]** Implement getAdjacentVertices() method – 2 marks
    Pass both testGetAdjacentVertices test cases – 2 marks
    Create and pass your own testGetNeighbourCustom() test case – 2 marks
  - **[14 Marks]** Implement depthFirstTraversal() method – 6 marks
    Pass both testDFT test cases – 4 marks
    Create and pass your own testDFTCustom() test case – 2 marks
    Pass another test case written by the TA – 2 marks
  - **[22 Marks]** Implement findShortestPath() method – 12 marks
    Pass both testFindShortestPath test cases – 6 marks
    Create and pass your own testFindShortestPathCustom() test case – 2 marks
    Pass another test case written by the TA – 2 marks
  - **[20 Marks]** Implement the buildGraph() method using the breadth-first search approach
    Pass test cases written by the TA

  - **Note:** If a test case fails, the test case marks are deducted, and only partial marks will be awarded to the implementation marks based on the severity of the logic error.

- **[Total 30 Marks] Lab Demo Section – Automated TileGraph Generation from TileMap**
  - **[6 Marks]** **(Code Walkthrough)** Custom algorithm for finding the neighbouring intersections (to the right and down) of a given Intersection
  - **[10 Marks]** **(Code Walkthrough)** Custom algorithm for building the graph based on a tile map.
  - **[10 Marks]** Discussion of the time and space complexity of the buildGraph() method. You need to identify the parameters the running time depends on, express the running time using the big-Oh notation and justify. Do the same for the space complexity.
  - **[4 Marks]** Demonstration of the resultant Automatic Path Generation and Tracer (P) following the generated path from the Start intersection to the Goal intersection.

System Overview

As you join the development team of this top-view city navigation game, the existing team members have already laid down the groundwork to support your automatic driver AI design using Graph and all the relevant algorithms. As illustrated in the figure, the game world is made up of the basic unit of Tile, which is represented in the console-screen prototype as a single character space. **Each Tile can be of the following kind: Occupied (#), Roads (space character), and Intersections (I)**. A random tile-map is automatically generated upon program bootup, where a complete road network connecting all intersections in the city is generated. The borders of the tile-map contain only occupied tiles.

For prototyping purpose, the starting intersection (S) is always on the top-left corner of the map, and the goal/destination intersection (D) is always on the bottom-right corner. In addition, all intersections on the map can be reached from S by going down or to the right. Along with the map objects, a Tracer (P) class is created as the self-driving AI that will generate its own navigation path to travel from the S to D. The Tracer is drawn on top of the map to indicate its current location.

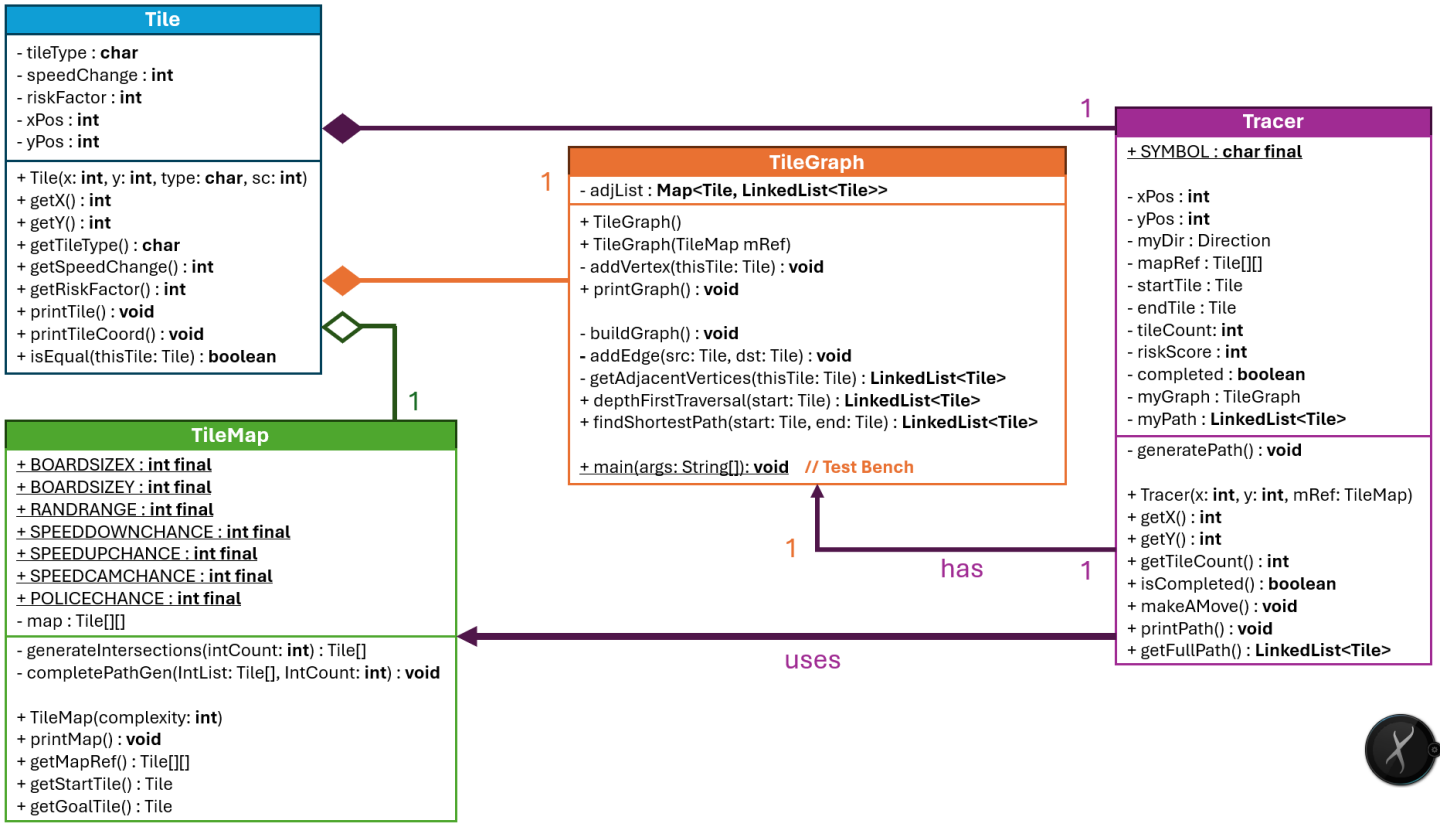
The following figure is a graphical illustration of a Tile Map.

START ROADS INTERSECTIONS

```
##################################################################################################
#S          I  I  I                                   I           I     I                  I######
# ########## ## ## ######################################### ########### ##### ################## ######
#I        I  I  I  I        I        I                    I     I     I##### ################## ######
# ####### ## ## ## ######## ######## #################### ##### ##### ##### ################## ######
# ####### ## ## ## ######## ######## #################### ##### ##### ##### ################## ######
# ####### ## ## ## ######## ######## #################### ##### ##### ##### ################## ######
# ####### ## ## ## ######## ######## #################### ##### ##### ##### ################## ######
# ####### ## ## ## ######## ######## #################### ##### ##### ##### ################## ######
# ####### ## ## ## ######## ######## #################### ##### #####I     I                  I    I#
# ####### ## ## ## ######## ######## #################### ##### ##### ##### ################## #### #
# ####### ## ## ## ######## ######## #################### ##### ##### ##### ################## #### #
#I        I  I## ## ########I        I           I        I     I     I##### ################## #### #
# ####### ## ## ## ######## ######## ########### ######## ##### ##### ##### ################## #### #
# ####### ## ## ## ######## ######## ########### ######## ##### ##### ##### ################## #### #
# ####### ## ## ## ######## ######## ########### ######## ##### ##### ##### ################## #### #
# ####### ## ## ## ######## ######## ########### ######## ##### ##### ##### ################## #### #
# ####### ## ## ## ######## ######## ########### ######## ##### ##### ##### ################## #### #
#I        I  I  I  I        I        I           I        I##### ##### ##### ################## #### #
# ########## ##### ######## #################### ######## ##### ##### ##### ################## #### #
# ########## ##### ######## #################### ######## ##### ##### ##### ################## #### #
# ##########I     I        I                    I        I     I     I     I              I  I    I#
# ########## ##### ################################################### #################### ####### #
# ########## ##### ################################################### #################### ####### #
#I          I     I                                                   I                   I       I#
############ ######################################################### #################### ####### #
############ ######################################################### #################### ####### #
############ ######################################################### #################### ####### #
############I                                                         P.......I.......................I.......D#
##################################################################################################
```

TRACER PATH GOAL / DESTINATION

The following UML diagram summarizes how Tile, TileMap, Tracer, and MainGame classes interact with each other. Your task is to complete the TileGraph class used by the Tracer class, so that the Tracer will hold the topological knowledge of the map, and can apply different path-finding algorithms in the future to find the shortest / most cost-effective route from S to D. This then will pave the way for the game development team for more enhanced AI designs in the next few design iterations.

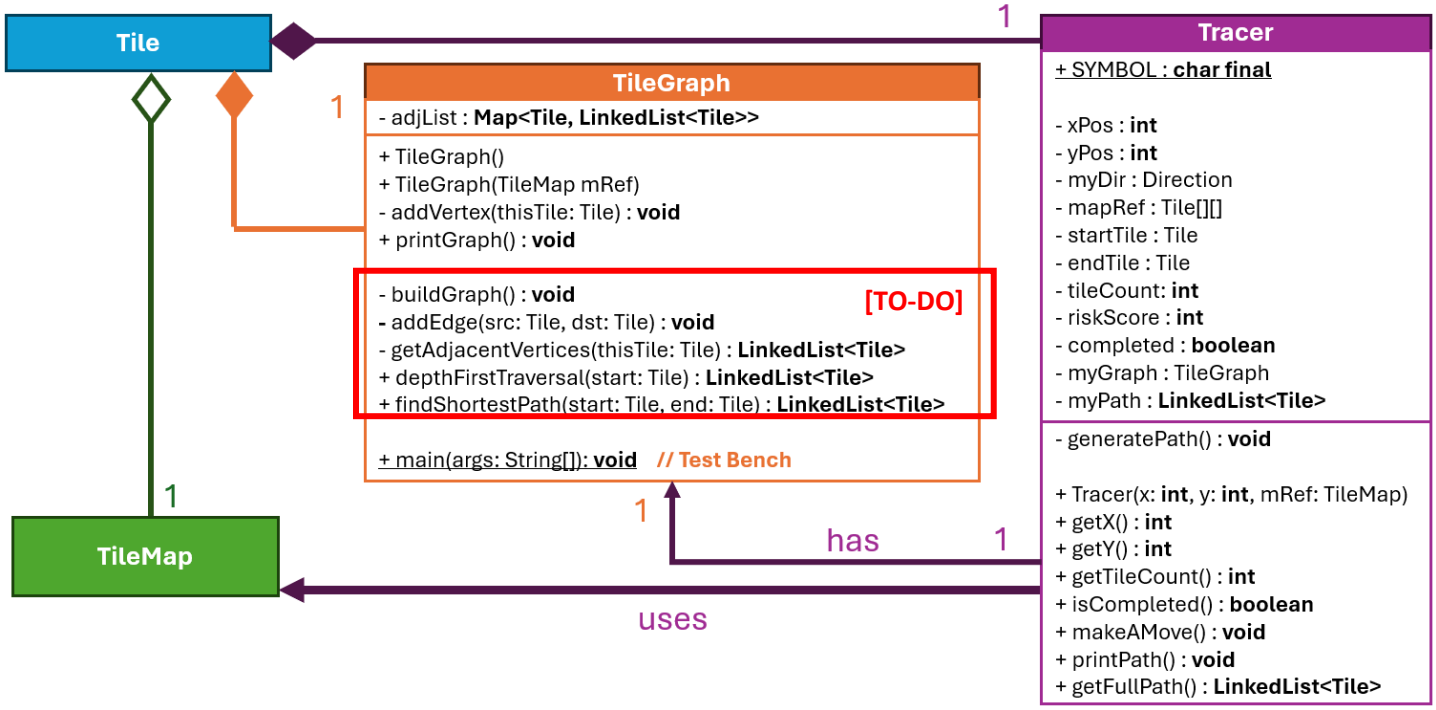


Your task is to complete the TileGraph implementations and thoroughly test it using the provided test bench.

TileGraph and Tracer Classes– Design Specifications

This section summarizes the design specifications of TileGraph class and of the Tracer class that require your design attention. All relevant and helper methods from other classes are highlighted here as well.

All methods that require your implementation will be tagged **[TO-DO]**.



I. TileGraph Members

- **private Map<Tile, LinkedList<Tile>>** adjList
  - A collection of pairs (Tile, LinkedList), where the first member is a vertex and the second member is the adjacency list of that vertex
  - Each vertex of the graph appears in only one pair
  - An object of type Map<K,V> is a collection of ordered pairs (key, value), where the key is an object of type K and the value is an object of type V
  - The key is now a Tile object representing a vertex in the graph
  - The value is now a LinkedList of Tile objects representing the adjacency list of the vertex
  - Use the methods specified in the interface Map<K,V> to operate on this collection
  - Read the documentation of the java built-in interface Map<K,V> at https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Map.html

- **public** TileGraph()
  - Default constructor of TileGraph; instantiates an empty graph

- **public** TileGraph(TileMap mRef)
  - Constructor: instantiates a graph representing the input map of tiles
  - It invokes the private method buildGraph () (which you have to implement) to populate the graph

- **private void** addVertex(Tile thisTile)
  - Adds thisTile as a new vertex in the graph, if it is not already there, with an empty adjacency list. In other words, it adds a new pair to the collection adjList, with the vertex being thisTile and the adjacency list empty.

- [TO-DO] **private void** addEdge(Tile src, Tile dst)
  - Add a ***directed*** edge from the source Tile (src) to the destination Tile (dst). The edge has to be added at the end of the adjacency list of the source Tile src.

- If the designated edge already exists, **<u>do not add</u>**.
- One line of code is given; you have to complete the rest. Use methods specified in the interface Map<K,V>.

- **[TO-DO]** **private void** buildGraph(TileMap mRef)
  - Adds vertices and edges to the existing empty graph to build the graph corresponding to mRef
  - Vertices are intersections; edges are roads connecting two consecutive intersections from left to right or from up to down
  - Edges are directed: from up to down or from left to right
  - Assumptions about the input TileMap:
    - All tiles on the border are occupied.
    - There is an intersection at the top left corner (at position [1][1] in the 2D Tile array).
    - Any intersection in the map can be reached from the top left intersection.
  - Must use an approach similar to Breadth-First Search (for efficiency) to get the maximum grade. This means that you have to start at the top left corner and discover all intersections and connecting roads between neighbouring intersections using a BFS approach. You may write additional private methods as helpers, for instance to find the neighbouring intersections (down or to the right) of a given intersection.

- **[TO-DO]** **private LinkedList<Tile>** getAdjacentVertices(Tile thisTile)
  - Returns the list of vertices adjacent to thisTile.

- **[TO-DO]** **public LinkedList<Tile>** depthFirstTraversal(Tile start)
  - Traverses the entire graph using the Depth-First Search algorithm.
  - Returns the list of the visited vertices in chronological order.
  - To record that a vertex was visited, while being able to check in constant time (on average) the "visited" status, you may use a HashSet object. Once a vertex is visited, add it to the HashSet. To find out if some vertex was visited search for it in the HashSet. You may use the built-in java class HashSet<E>. The documentation can be found at: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/HashSet.html

- **[TO-DO]** **public LinkedList<Tile>** findShortestPath(Tile start, Tile end)
  - Finds the unweighted shortest-path from the start tile to the end tile, in other words, **<u>the path with the smallest number of edges/vertices.</u>**
  - **The method should return the list of vertices on the unweighted shortest path in reversed order (i.e., the first vertex in the list is the last vertex on the path, the second vertex in the list is the second last on the path and so on).**
  - You should use the **<u>Breadth-First Searching Algorithm</u>**, and record the "parent" of each vertex visited + its "distance" from the start tile, until the end tile is encountered.
  - For the queue needed in the algorithm, you may use the built-in java class LinkedList<E>. The documentation can be found at: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/HashSet.html
  - To record that a vertex was visited, while being able to check in constant time (on average) the "visited" status, you may use a HashSet object. Once a vertex is visited, add it to the HashSet. To find out if some vertex was visited search for it in the HashSet. You may use the built-in java class HashSet<E>. The documentation can be found at: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/HashSet.html
  - To store the "parent" of each vertex (i.e., the variable **v.prev**), you may use a HashMap object created with the built-in java class HashMap<K,V>, with K=Tile and V=Tile. See the documentation at: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/HashMap.html

- To store the distance of each node (which is the number of edges from the start), you may use a HashMap object created with the built-in java class HashMap<K,V>, with K=Tile and V=Integer. See the documentation at: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/HashMap.html

- **public void** printGraph()
  - Prints each vertex, followed by its adjacency list.

II. Useful Fields and Methods in the Tracer Class

Only the Lab4-related members are covered here.

- **private** Tile[][] mapRef
  - The reference to the TileMap in the game, in the format of a 2D Tile array. This is used to build the myGraph member of the Tracer class.

- **private** Tile startTile
- **private** Tile endTile
  - The Tile of the Start Intersection (startTile) and the Goal / Destination Intersection (endTile)
  - You need to use these two Tile instances to drive myGraph member to generate the shortest path from startTile to endTile with the least number of intersections along the path.

- **private** TileGraph myGraph
  - The TileGraph object that represents the graph constructed based on the TileMap.
  - myGraph is initialized by using the second constructor of TileGraph

- **private** LinkedList<Tile> myPath
  - Holds the unweighted shortest path that the Tracer instance should follow to reach the destination tile.
  - Updated through invoking findShortestPath() on myGraph, which will return the path in the format of LinkedList<Tile> containing all the intersections along the shortest path.
  - This member is returned by the getFullPath() method from the Tracer class to the main game for drawing the Path as ASCII period character (.) on the TileMap to indicate the path to be followed by the Tracer instance.

- **public void** printPath()
  - Prints all the Tiles in the myPath member to help you verify whether your shortest path is correctly generated, and that your myGraph member is correctly generated.

III. Useful Methods from Tile Class

While you may find other methods in the Tile class also applicable to your design, these are the methods that will certainly be helpful in debugging and code development. Use them wherever you see fit.

- **public int** getX()
- **public int** getY()
- **public char** getTileType()

- **public void** printTile()
    - Print the tile type (character) to be displayed on the game board.

- **public void** printTileCoord()
    - Print the tile x-y coordinate.
    - Use this method to differentiate different vertices in the graph.

IV. Useful Java Container Classes

Using the existing Java container classes is allowed in this lab, such as:

- Map<K,V> / HashMap<E>
- Set<E> / LinkedHashSet<E>, HashSet<E>
- Stack<E>
- List<E> / LinkedList<E>
- Collection<E>
- Iterator<E>

They are defined in the package java.util in module java.base. The documentation can be found at: https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/package-summary.html

Example: If you need a linked list of Tile objects in your code, you can use:

LinkedList<Tile> list = new LinkedList<Tile>(); // this line declares the reference variable "list", instantiates a LinkedList object and makes "list" point to this object.

Now "list" is a reference to an empty list of Tiles. You can add items to the list or perform other operations on the list by using methods from the class LinkedList<E>.